



Scripting Reference

Scripts can be found and created in different places of Chataigne. Depending on where they are executed, different methods and callbacks are available. Here are the different script types :

- ★ [Module Scripts](#)
- ? [Condition Scripts](#)
- ! [Consequence Script](#)
- ⌘ [Mapping Filter Scripts](#)
- 🚀 [Mapping Output Script](#)

💡 When creating a script, the generated file will be pre-filled with contextual methods and functions depending on where it has been created. This allows for easily starting a new script with already relevant methods and callbacks !

Common Reference

[Common Functions](#)

[Parameters and triggers](#)

[Common methods and properties](#)

[Trigger](#)

[Float Parameter](#)

[Integer Parameter](#)

[Boolean Parameter](#)

[String Parameter](#)

[Color Parameter](#)

[Target Parameter](#)

[Enum Parameter](#)

[File Parameter](#)

[Point2D Parameter](#)

[Point3D Parameter](#)

[Container](#)

[Manager](#)

[States](#)

[Automation](#)

[Commands](#)

[Script object](#)

[Root object](#)

[Local object](#)

[Util object](#)

Common Reference

This is the common reference for all scripts, no matter their type

Here you can find an almost exhaustive reference on Chataigne specific functions.

💡 Scripts in Chataigne are based on a simplified version of Javascript. Most functions are available, but some might be missing. If you're missing some functions, please contact me and consider [donating](#) so I can spend more time improving Chataigne !

💡 This documentation only shows functions that I have added on top of JUCE's Javascript Engine. To know all the base functions that are available, you can take a look at the [JavascriptEngine's source code](#)

Common Functions

There are few common functions that you can use, no matter the type of script.

💡 None of the functions below are needed when making a script, they are just helper functions.
If you don't need them, don't add them in your script as Chataigne will optimize your script depending on the available functions in it.

Method	Description	Example
<code>init()</code>	If present, this function will be called right after the script is loaded.	<pre>function init() { //init your values, log things and assign variables }</pre>
<code>update(<i>deltaTime</i>)</code>	If present, this function will be called regularly at rate specified by the "Update rate" script parameter. This parameter is only visible when the function is present in the script. You can also change the rate from the script by calling <code>script.setUpdateRate(<i>rate</i>)</code> . see below for more informations. <i>deltaTime</i> is in seconds, and the <i>rate</i> parameter is in Hz.	<pre>function update(deltaTime) { script.log("Delta time : " + deltaTime); }</pre>
<code>messageBoxCallback(<i>id</i>, <i>result</i>)</code>	This function is called when a user has made a choice in a message box launched by this script.	<pre>function messageBoxCallback(id, result) {script.log("Message box callback : "+id+" > "+result); }</pre>

Parameters and triggers

All parameters and triggers have common methods and specific methods

Common methods and properties

Method	Description	Example
<code>getParent()</code>	Returns the parent of this parameter	<code>myParam.getParent();</code>
<code>setName(name)</code>	Sets the parameters name	<code>myParam.setName("new name");</code>
<code>setAttribute(attribute, value)</code>	Sets a parameter's attribute. All parameters get : <i>readonly</i> : sets the parameter as not editable <i>description</i> : change the tooltip description <i>enabled</i> : enable or disable this parameter <i>alwaysNotify</i> : set the alwaysNotify flag (if true, this will trigger updates even if setting the same value than before). More information for specific parameters in each parameter's section.	<code>myParam.setAttribute("readonly",true);</code> <code>myParam.setAttribute("description"," new description");</code> <code>myParam.setAttribute("enabled",false);</code> <code>myParam.setAttribute("alwaysNotify", true);</code>
<code>isParameter()</code>	returns whether the target is a parameter or a trigger	<code>var isParam = myTarget.isParameter();</code>
<code>name</code>	This is the name identifier of the object, as you would target it in a script.	<code>script.log(myParam.name);</code>
<code>niceName</code>	This is the "nice name" of the object, as you see it in the Inspector.	<code>script.log(myParam.niceName);</code>
<code>getControlAddress</code> ([reference])	Returns the OSC-style control address of this element. Optional <i>reference</i> parameter allows you to specify a root container to generate the address from.	<code>script.log(myParam.getControlAddress());</code>
<code>getScriptControlAddress()</code>	Returns the script control address of this element.	<code>script.log(myParam.getScriptControlAddress());</code>
<code>resetValue()</code>	Resets the value to its default	<code>myParam.resetValue();</code>
<code>getJSONData()</code>	Returns the raw JSON data for this parameter	<code>script.log(myParam.getJSONData());</code>
<code>loadJSONData(data)</code>	Load raw JSON data to this parameter (as returned by <code>getJSONData()</code> or from a saved file)	<code>param1.loadJSONData(param2.getJSONData());</code>

Trigger

Method	Description	Example
<code>trigger()</code>	Returns the value of this parameter	<code>myTrigger.trigger();</code>

Float Parameter

Method	Description	Example
<code>get()</code>	Returns the value of this parameter	<code>var value = myFloatParam.get();</code>
<code>set(value)</code>	Sets the value of this parameter	<code>myFloatParam.set(.5);</code>
<code>setAttribute(attribute, value)</code>	Specific attributes : <i>ui</i> : The UI to use to show this parameter. Accepted values are : <i>time</i> , <i>slider</i> , <i>stepper</i> , <i>label</i> <i>unitSteps</i> : Which step size to use for this parameter. Set to 0 for free modification <i>stringDecimals</i> : How many decimals to use when converting to String	<code>myStringParam.setAttribute ("ui", "time");</code>

Integer Parameter

Method	Description	Example
<code>get()</code>	Returns the value of this parameter	<code>var value = myIntParam.get();</code>
<code>set(value)</code>	Sets the value of this parameter	<code>myIntParam.set(2);</code>
<code>setAttribute(attribute, value)</code>	Specific attributes : <i>hexMode</i> : Whether to show the value as decimal or hexadecimal.	<code>myStringParam.setAttribute ("hexMode", true);</code>

Boolean Parameter

Method	Description	Example
<code>get()</code>	Returns the value of this parameter	<code>var value = myBoolParam.get();</code>
<code>set(value)</code>	Sets the value of this parameter	<code>myBoolParam.set(true);</code>

String Parameter

Method	Description	Example
<code>get()</code>	Returns the value of this parameter	<code>var value = myStringParam.get();</code>
<code>set(value)</code>	Sets the value of this parameter	<code>myStringParam.set("super");</code>
<code>setAttribute(attribute, value)</code>	Specific attributes : <i>multiline</i> : whether the parameter can be multiline <i>prefix</i> : add a prefix before the value <i>suffix</i> : add a suffix after the value	<code>myStringParam.setAttribute ("multiline", true);</code>

Color Parameter

Method	Description	Example
<code>get()</code>	Returns the value of this parameter. The result is an array containing [r,g,b,a], values are between 0 and 1	<code>var value = myColorParam.get();</code> <code>script.log("green : "+value[1]);</code>
<code>set(value)</code>	Sets the value of this parameter. You can either set the value with ARGB hex value, [r,g,b,a] float array, or r,g,b,a separate values. r, g and b values are always between 0 and 1 and alpha is optional (you can specify r,g,b only);	<code>//this is all set to cyan alpha 100%</code> <code>myColorParam.set(0xff00ffff);</code> <code>myColorParam.set([0,1,1,1]);</code> <code>myColorParam.set(0,1,1,1);</code>

Target Parameter

Method	Description	Example
get()	Returns the string value of the target address	<code>var address = myTargetParam.get();</code>
getTarget()	Returns the actual selected target	<code>var target = myTargetParam.getTarget();</code>
set(value)	Sets the string value of the target's address.	<code>myTargetParam.set ("/root/modules/osc");</code>
setAttribute(attribute, value)	Specific attributes : targetType : the type of target to search. This can either be " container " or " controllable " root : the root container from which to search searchLevel : specify the max level to search from the root showParameters : if target is controllable, hide or show parameters showTriggers : if target is controllable, hide or show triggers labelLevel : the level of parents to show in the UI. allowedTypes : specify which types of targets are allowed excludedTypes : specify which types of targets are not allowed	<code>myTargetParam.setAttribute ("searchLevel",2);</code> <code>myTargetParam.setAttribute ("root", root.sequences);</code> <code>myTargetParam.setAttribute("allowedTypes", "Mapping");</code>

Enum Parameter

Method	Description	Example
addOption(label, value)	Add an option to the list	<code>myEnumParam.addOption("new option",3);</code>
get()	1.6.x : Return the selected key 1.7.x : Return the selected data	<code>var label = myEnumParam.get(); //1.6.x</code> <code>var value= myEnumParam.get(); //1.7.x</code>
getData()	Get the selected data (1.6.x only)	<code>var data = myEnumParam.getData(); //1.6</code>
getKey()	Get the selected key (1.7.x only)	<code>var key = myEnumParam.getKey(); //1.7</code>
removeOptions()	Removes all options.	<code>myEnumParam.removeOptions();</code>
set(key)	This sets the parameter to the corresponding key.	<code>myEnumParam.set("Option 1");</code>
setData(data)	This sets the parameter to the corresponding data	<code>myEnumParam.setData("data1");</code>
setNext([loop])	Sets the value to the next element in list. Loop (optional) : loops throught the list	<code>myEnumParam.setNext(true);</code>
setPrevious([loop])	Sets the value to the previous element in list. Loop (optional) : loops throught the list	<code>myEnumParam.setPrevious(true);</code>
getAllOptions()	Returns an array with all available option. Each element of the array is an object containing a "key" and a "value" property.	<code>var options = myEnumParam.getAllOptions();</code> <code>script.log(options[0].key);</code>
getOptionAt(index)	Return an option at the defined index . This object contains a "key" and a "value" property.	<code>var options = myEnumParam.getAllOptions();</code> <code>script.log(options[0].key);</code>
getIndex()	Returns the index of the selected option.	<code>var index = myEnumParam.getIndex();</code>

File Parameter

Method	Description	Example
setAttribute (<i>attribute, value</i>)	Specific attributes : directoryMode : specify if this parameter should be looking for a file or a folder saveMode : specify if this parameter should open or save a file autoReload : specify if the contents of this file should be automatically reloaded from the filesystem	<pre>myFileParam.setAttribute ("directoryMode",true);</pre>
readFile (<i>[asJSON]</i>)	Get the content of the file. If asJSON is <i>true</i> , then the content will be parsed and returned as an Object.	<pre>var myTextContent = myFileParam.readFile (); var myJSONContent = myFileParam.readFile (true);</pre>
writeFile (<i>data, overwriteIfExists</i>)	Write the content or a string of an Object to the selected file. If the file exists, overwriteIfExists will let you decide whether to overwrite it or not (false by default).	<pre>var data = "super"; myFileParam.writeFile (data, true);</pre>
getAbsolutePath ()	Returns the absolute path for this file.	<pre>var path = myFileParam.getAbsolutePath();</pre>
launchFile (<i>arguments**</i>)**	This will launch the selected file with the _arguments _ provided	<pre>myFileParam.launchFile("--verbose")</pre>

Point2D Parameter

Method	Description	Example
get ()	Returns the value of this parameter	<pre>var value = myP2DParam.get();</pre>
set (<i>x, y</i>)	Sets the value of this parameter	<pre>myP2DParam.set(.5, 2);</pre>
setAttribute (<i>attribute, value</i>)	Specific attributes : ui : The UI to use to show this parameter. Accepted values are : time, slider, stepper, label canvasInvertX : Invert X direction for canvas editor (when the P2D has a range) canvasInvertY : Invert Y direction for canvas editor canvasStretchMode : Stretch canvas editor stringDecimals : How many decimals to use when converting to String	<pre>myP2DParam.setAttribute ("ui", "time");</pre>

Point3D Parameter

Method	Description	Example
get ()	Returns the value of this parameter	<pre>var value = myP3DParam.get();</pre>
set (<i>x, y, z</i>)	Sets the value of this parameter	<pre>myP3DParam.set(.5, 2, -1);</pre>
setAttribute (<i>attribute, value</i>)	Specific attributes : ui : The UI to use to show this parameter. Accepted values are : time, slider, stepper, label stringDecimals : How many decimals to use when converting to String	<pre>myP3DParam.setAttribute ("ui", "time");</pre>

Container

Containers are any object that contains parameters or other containers. If you're targetting something from the hierarchy (*root.****** or *local.**), this will be either a container of a parameter, so if it's not a parameter, then it's a container.

Method	Description	Example
<code>getChild(<i>childName</i>)</code>	Returns the parameter or container with the provided name.	<code>var child = myContainer.getChild("activity");</code>
<code>getParent()</code>	Returns the parent container	<code>var parent = myContainer.getParent();</code>
<code>setName(<i>name</i>, [<i>shortName</i>])</code>	Changes the name of the container. If <i>shortName</i> is specified, it will set a custom short name that you can access easily from the script.	<code>myContainer.setName("new name");</code>
<code>setCollapsed(<i>value</i>)</code>	Change the collapsed state of the container, if it can be collapsed.	<code>myContainer.setCollapsed(false);</code>
<code>name</code>	This is the name identifier of the object, as you would target it in a script.	<code>script.log(myContainer.name);</code>
<code>niceName</code>	This is the "nice name" of the object, as you see it in the Inspector.	<code>script.log(myContainer.niceName);</code>
<code>addTrigger(<i>name</i>, <i>description</i>)</code>	This will add a trigger (button).	<code>var myTrigger = myContainer.addTrigger("My Trigger", "Trigger description");</code>
<code>addFloatParameter(<i>name</i>, <i>description</i>, <i>default</i>, <i>min</i> <i>max</i>)</code>	This will add a float number parameter (slider).	<code>var myFloatParam = myContainer.addFloatParameter("My Float Param", "Description of my float param", .1, 0, 1);</code>
<code>addIntParameter(<i>name</i>, <i>description</i>, <i>default</i>, <i>min</i> <i>max</i>)</code>	add an integer number parameter (stepper), default value of 2, with a range between 0 and 10	<code>var myIntParam = myContainer.addIntParameter("My Int Param", "Description of my int param", 2, 0, 10);</code>
<code>addBoolParameter(<i>name</i>, <i>description</i>, <i>default</i>)</code>	add a boolean parameter (toggle)	<code>var myBoolParam = myContainer.addBoolParameter("My Bool Param", "Description of my bool param", false);</code>
<code>addStringParameter(<i>name</i>, <i>description</i>, <i>default</i>)</code>	add a string parameter (text field)	<code>var myStringParam = myContainer.addStringParameter("My String Param", "Description of my string param", "cool");</code>
<code>addColorParameter(<i>name</i>, <i>description</i>, <i>default</i>)</code>	add a color parameter (color picker). You can either set the default color with ARGB hex value, [r,g,b,a] float array, or r,g,b,a separate values. r, g and b values are always between 0 and 1 and alpha is optional (you can specify r,g,b only);	<code>var myColorParam = script.addColorParameter("My Color Param", "Description of my color param", 0xff0000ff); //default blue alpha 100% var myColorParam2 = script.addColorParameter("My Color Param 2", "Description of my color param", [1, 0, 1]); //default purple</code>
<code>addPoint2DParameter(<i>name</i>, <i>description</i>)</code>	add a point 2d parameter	<code>var myP2DParam = myContainer.addPoint2DParameter("My P2D Param", "Description of my p2d param");</code>
<code>addPoint3DParameter(<i>name</i>, <i>description</i>)</code>	add a point 3d parameter	<code>var myP3DParam = myContainer.addPoint3DParameter("My P3D Param", "Description of my p3d param");</code>
<code>addTargetParameter(<i>name</i>, <i>description</i>)</code>	add a target parameter (to reference another parameter)	<code>var myTargetParam = myContainer.addTargetParameter("My Target Param", "Description of my target param");</code>
<code>addFileParameter(<i>name</i>, <i>description</i>, [directoryMode])</code>	Add a file parameter (to reference file or folder on the disk). If directoryMode is set to true a folder instead of a file can be selected.	<code>var myFileParam = myContainer.addFileParameter("My File Param", "Description of my file param");</code>
<code>addEnumParameter(<i>name</i>, <i>description</i>, <i>label1</i>, <i>value1</i>, <i>label2</i>, <i>value2</i>, ...)</code>	Add a enum parameter (dropdown with options) Each pair of values after the first 2 arguments define an option and its linked data	<code>var myEnumParam = myContainer.addEnumParameter("My Enum Param", "Description of my enum param", "Option 1", 1, "Option 2", 5, "Option 3", "banana");</code>
<code>addContainer(<i>name</i>)</code>	Add a container inside of the container.	<code>var childContainer = myContainer.addContainer("My Child Container");</code>
<code>addAutomation(<i>name</i>)</code>	Adds a curve container	<code>var automation = myContainer.addAutomation("Curve");</code>
<code>removeContainer(<i>name</i>)</code>	Remove a child container from the container.	<code>myContainer.removeContainer ("myChildContainer")</code>
<code>removeParameter(<i>name</i>)</code>	Remove a parameter from the container.	<code>myContainer.removeParameter ("myChildParameter");</code>
<code>getControlAddress</code> ([reference])	Returns the OSC-style control address of this element. Optional <i>reference</i> parameter allows you to specify a root container to generate the address from.	<code>script.log(myContainer.getControlAddress());</code>
<code>getScriptControlAddress()</code>	Returns the script control address of this element.	<code>script.log(myContainer.getScriptControlAddress());</code>
<code>getContainers()</code>	Returns an array containing all the child containers for this container. This is a good way of ensuring that you get an up-to-date list of all children containers for an object.	<code>var children = myContainer.getContainers();</code>
<code>getControllables</code> ([<i>includeParameters</i> = true], [<i>includeTriggers</i> = true])	Returns an array containing all the child parameters and triggers for this container. This is a good way of ensuring that you get an up-to-date list of all children containers for an object.	<code>var childParams = myContainer.getControllables(true, false); var childTriggers = myContainer.getControllables(false, true);</code>

	The two optional paramerters filter the type of controllables you want to get.	
<code>clear([clearContainers = true], [clearControllables = true])</code>	Removes all child containers and controllables for this container (depending on the arguments passed)	<code>myContainer.clear();</code>
<code>getJSONData()</code>	Returns the raw JSON data for this container	<code>script.log(myContainer.getJSONData());</code>
<code>loadJSONData(data)</code>	Load raw JSON data to this container (as returned by getJSONData() or from a saved file)	<code>container1.loadJSONData(container2.getJSONData());</code>

Manager

Managers are special types of container. In the software, you can see that a container is a manager when there is a "+" icon on the right of its block. Well, almost all the times, some of them are just enhanced containers. Most of the times, the managers you will be interested in are the **Module manager, the Sequence manager, Layer managers and Automation keys managers**.

When a manipulating a manager, you have access to specific functions and properties like adding items, removing items or getting an array of it's items.

Method	Description	Example
<code>addItem([type])</code>	Adds an item to the manager and returns it. Some managers like the Module manager or the layer manager can create multiple types of items. Thus they need a type of item to create, which you must set in the type argument. Other managers like the Sequence manager don't need this argument because there is only one type of sequence. In these cases you don't need to provide an argument when calling the function.	<code>var newOSCModule = root.modules.addItem("OSC");</code> <code>var newSequence = root.sequences.addItem();</code>
<code>removeItem(item)</code>	Removes an item. item can be either the item object or a string representing its name (this will search for exact match of niceName, shortName, and lowercase version)	<code>root.modules.removeItem (myModule);</code> <code>root.modules.removeItem ("My Module");</code>
<code>removAll(item)</code>	Removes all items. More effective since it handles it all at once, also for Undo/Redo	<code>root.modules.removeAll();</code>
<code>getItems()</code>	Returns an array containing all the items of this manager.	<code>var items = root.sequences.getItems();</code> <code>script.log("Num sequences : "+items.length);</code>
<code>getItemWithName(name)</code>	Returns the first item found with name . This will search for exact match of niceName, shortName, and lowercase version.	<code>var introSeq = root.sequences.getItemWithName ("Intro");</code>
<code>getItemAt(index)</code>	Returns the item at the index .	<code>var curSequence = root.sequences.getItemAt(1); //0 is first, so this will get the second sequence in the list.</code>
<code>getItemIndex(item)</code>	Returns the index of the specified item .	<code>var index= root.sequences.getItemIndex (curSequence);</code>
<code>getItemBefore(item)</code>	Returns the item before the specified item .	<code>var prevSequence = root.sequences.getItemBefore (curSequence);</code>
<code>getItemAfter(item)</code>	Returns the item after the specified item .	<code>var nextSequence = root.sequences.getItemAfter (curSequence);</code>
<code>reorderItems()</code>	Reorders all items in this manager.	<code>var mappingLayer = root.sequences.sequence.layers.mapping;</code> <code>mappingLayer.automation.reorderItems();</code>

States

States have dedicated methods to ease the creations of certain items like transitions.

Method	Description	Example
<code>addTransition(source, dest)</code>	Adds a transition between two states. source and dest are the two names of the States to transition from/to	<code>var transition = root.states.addTransition("State A", "State B");</code>

Automation

Automations are special managers that handle curves. You can find them in Mapping Layers, or Curve Map filters for instance. They have special methods that you can use to get more specialized info about the curve data.

Method	Description	Example
<code>addKey(<i>position</i>, <i>value</i>)</code>	Adds a key at the given position and value	<code>var key = automation.addKey(.5,1);</code>
<code>setLength(<i>length</i>, <i>stretch</i>, <i>stickToEnd</i>)</code>	Set a new length for this automation. Stretch allows to stretch all keys to keep relative positioning across the whole automation, stickToEnd forces them to repositioning relative to then end	<code>automation.setLength(12.1, false, false);</code>
<code>getValueAtPosition(<i>position</i>)</code>	Returns the value of the curve at the given position	<code>var val = automation.getValueAtPosition(2.5);</code>
<code>getKeyAtPosition(<i>position</i>)</code>	Returns the closest key at the given position	<code>var key = automation.getKeyAtPosition(2.5);</code>
<code>getKeysBetween(<i>start</i>, <i>end</i>)</code>	Returns an array of all the keys contained between <i>start</i> and <i>end</i>	<code>var keys = automation.getKeysBetween(2.5, 4.7);</code>

Commands

Commands are used to output data from Mappings, Multiplexes or Mapping Layers for example. They have special methods used for managing the links to the related variables (for example, if a mapping has multiple inputs, links let you specify which of the inputs should go to the Output command).

Method	Description	Example
<code>linkParamToMappingIndex(<i>parameter</i>, <i>index</i>)</code>	Links <i>parameter</i> to the mapping index <i>index</i> (starting at 1). <i>parameter</i> may be an object or a script control address.	<code>root.states.state.processors.mapping.outputs.mappingOutput.command. .linkParamToMappingIndex("value", 1);</code>
<code>setParamLink(<i>parameter</i>, <i>linkDef</i>)</code>	Create a new link for <i>parameter</i> according to <i>linkDef</i> (useful for more complex use cases) <i>linkDef</i> must be an object containing a "type" property and a "value" property. The following types are accepted for the “type” property : - input : create a “mapping input” link ("value" sets the mapping index, starting at 1) - index : create a “multiplex index” link, starting at 1 ("value" is unused) - index0 : create a “multiplex index” link, starting at 0 ("value" is unused) - list : create a “multiplex list” link ("value" sets the list name using its script control address) <i>parameter</i> may be an object or a script control address	<code>root.states.state.processors.multiplex.processors.mapping.outputs. mappingOutput.command.setParamLink("value", {"type":"index", "value":""});</code> <code>root.states.state.processors.multiplex.processors.mapping.outputs. mappingOutput.command.setParamLink("value", {"type":"list", "value":"floatList"});</code>
<code>unlinkParam(<i>parameter</i>)</code>	Unlink <i>parameter</i> (may be an object or a script control address)	<code>root.states.state.processors.mapping.outputs.mappingOutput.command. .unlinkParam("value");</code>

Script object

The script object refers to the script container. You can add your own custom parameters here, as well as logging informations, warnings and errors.

 The add[...] functions here are overriding the ones from the Container object. Adding parameters in a script using script.add[...] is different in the way that reloading the script will force a rebuild of those parameters. The script is effectively managing the lifetime of those parameters, whereas a parameter created in a container with myContainer.add[...] will need to be handled manually.

Method	Description	Example
getScriptDirectory()	Returns the absolute path of the calling script's parent directory	<code>script.log(script.getScriptDirectory());</code>
getScriptPath()	Returns the absolute path of the calling script file	<code>script.log(script.getScriptPath());</code>
addTrigger (<i>name, description</i>)	This will add a trigger (button)	<code>var myTrigger = script.addTrigger("My Trigger", "Trigger description");</code>
addFloatParameter (<i>name, description, default, min max</i>)	This will add a float number parameter (slider).	<code>var myFloatParam = script.addFloatParameter("My Float Param","Description of my float param",.1,0,1);</code>
addIntParameter (<i>name, description, default, min max</i>)	add an integer number parameter (stepper), default value of 2, with a range between 0 and 10	<code>var myIntParam = script.addIntParameter("My Int Param","Description of my int param",2,0,10);</code>
addBoolParameter (<i>name, description, default</i>)	add a boolean parameter (toggle)	<code>var myBoolParam = script.addBoolParameter("My Bool Param","Description of my bool param",false);</code>
addStringParameter (<i>name, description, default</i>)	add a string parameter (text field)	<code>var myStringParam = script.addStringParameter("My String Param","Description of my string param", "cool");</code>
addColorParameter (<i>name, description, default</i>)	add a color parameter (color picker). You can either set the default color with ARGB hex value, [r,g,b,a] float array, or r,g,b,a separate values. r, g and b values are always between 0 and 1 and alpha is optional (you can specify r,g,b only);	<code>var myColorParam = script.addColorParameter("My Color Param","Description of my color param",0xff0000ff); //default blue alpha 100% var myColorParam2 = script.addColorParameter("My Color Param 2","Description of my color param",[1,0,1]); //default purple</code>
addPoint2DParameter (<i>name, description</i>)	add a point 2d parameter	<code>var myP2DParam = script.addPoint2DParameter("My P2D Param","Description of my p2d param");</code>
addPoint3DParameter (<i>name, description</i>)	add a point 3d parameter	<code>var myP3DParam = script.addPoint3DParameter("My P3D Param","Description of my p3d param");</code>
addTargetParameter (<i>name, description</i>)	add a target parameter (to reference another parameter)	<code>var myTargetParam = script.addTargetParameter("My Target Param","Description of my target param");</code>
addFileParameter (<i>name, description, [directoryMode = false]</i>)	Add a file parameter (to reference file or folder on the disk). If <i>directoryMode</i> is set to true a folder instead of a file can be selected.	<code>var myFileParam = script.addFileParameter("My File Param","Description of my file param");</code>
addEnumParameter (<i>name, description, label1, value1, label2, value2, ...</i>)	add an enum parameter (dropdown with options) Each pair of values after the first 2 arguments define an option and its linked data	<code>var myEnumParam = script.addEnumParameter("My Enum Param","Description of my enum param", "Option 1", 1,"Option 2", 5, "Option 3", "banana");</code>
log (<i>message</i>)	Logs a message (must activate "Log" in the script parameters)	<code>script.log("This is a message");</code>
logWarning (<i>message</i>)	Logs a message as a warning	<code>script.logWarning("This is a warning");</code>
logError (<i>message</i>)	Logs a message as an error	<code>script.logError("This is an error");</code>
setUpdateRate (<i>rate</i>)	Sets the rate at which the update() function is called	<code>script.setUpdateRate(50);</code>
setExecutionTimeout (<i>timeInSeconds</i>)	Sets the maximum time that a call in the script can take in seconds.	<code>script.setExecutionTimeout(10);</code>
refreshEnvironment()	This will resync the root, local and other potential environment objects. This is useful if you want to ensure that you're not accessing unsynced deleted objects.	<code>script.refreshEnvironment();</code>
function scriptParameterChanged (<i>param</i>)	This function will be called each time a parameter of this script has changed.	<code>function scriptParameterChanged(param){ script.log("Param changed : "+param.name); }</code>

Root object

The root object refers to Chataigne's engine, which is the root object of all parent.
It allows you to access any object in Chataigne's hierarchy. The best way to access them is to right click on a parameter's UI and select "Copy Script control address". Then you can past the address in your script and you will be able to control this parameter.

Local object

The local object depends on where the scripts is running.

- If the script is running inside a module, the local variable will referring to the module. You can find all the module functions in the [Module Scripts](#) section.

- If the script is running inside a condition, the local variable will be referring to the condition. You can find all the condition functions in the [Condition Scripts](#) section.
- If the script is running inside a filter, the local variable will be referring to the filter. You can find all the filter functions in the [Filter Scripts](#) section.

Util object

The util object provides helpers and utility functions like time or conversion.

Method	Description	Example
<code>getAppVersion()</code>	Returns the current version of the app	<code>var version = util.getAppVersion();</code>
<code>getOSInfos()</code>	Returns informations about the OS. The returned object contains the following properties : "name" : name of the OS "type": OS type "computerName": name of the computer language": OS language "username" : name of the logged on user	<code>var infos = util.getOSInfos(); script.log("Hello "+infos.username);</code>
<code>getEnvironmentVariable(key)</code>	Returns the value of an environment variable from the OS.	<code>var pathEnv = util.getEnvironmentVariable("PATH");</code>
<code>getTime()</code>	Returns the time since system start in seconds	<code>var time = util.getTime();</code>
<code>getTimestamp()</code>	Returns the time since January 1st 1970 in seconds	<code>var timestamp = util.getTimestamp();</code>
<code>delayThreadMS(milliseconds)</code>	delays the scripts execution the amount of milliseconds provided.	<code>util.delayThreadMS(200);</code>
<code>getFloatFromBytes(byte1, byte2, byte3, byte4)</code>	Returns a float from 4 bytes (big endian, byte1 is most significant)	<code>var value = util.getFloatFromBytes(0, 0, 2, 10);</code>
<code>floatToHexSeq(float, true)</code>	Returns a Sequence of HEX values from float (4 bytes) (big endian or little endian)	<code>var strValue = util.floatToHexSeq(15.1225,true);</code>
<code>getInt32FromBytes(byte1, byte2, byte3, byte4)</code>	Returns a 32-bit integer from 4 bytes (big endian, byte1 is most significant)	<code>var value = util.getInt32FromBytes(0, 0, 2, 10);</code>
<code>getInt64FromBytes(byte1, byte2, byte3, byte4, byte5, byte6, byte7, byte8)</code>	Returns a 64-bit integer from 8 bytes (big endian, byte1 is most significant)	<code>var value = util.getInt64FromBytes(0, 5, 7, 22, 0, 0, 2, 10);</code>
<code>doubleToHexSeq(double, true)</code>	Returns a Sequence of HEX values from double (8 bytes) (big endian or little endian)	<code>var strValue = util.doubleToHexSeq(15.1225,true);</code>
<code>colorToHex(r, g, b [,a])</code> <code>colorToHex([r, g, b [,a]])</code>	Returns a hex representation of the color, without the # Arguments can either be provided separate or inside an array If an alpha channel is provided, the resulting string will be of the form AARRGGBB, otherwise RRGGBB R, G and B are values between 0 and 1	<code>var strValue = util.colorToHex([1.0,0.3,.8]);</code>